

Code Explanation

Code Explanation: AWS Glue PySpark ETL Job - Customer Order Pipeline

Overview

This AWS Glue job implements a complete ETL pipeline that ingests customer and order data from S3, performs data quality transformations, joins the datasets, implements slowly changing dimensions (SCD Type 2) using Apache Hudi, and generates customer spending analytics.---

Step-by-Step Breakdown

Step 1: Import Dependencies

```
import sys, os
from datetime import datetime
from pyspark.context import SparkContext
from pyspark.sql import SparkSession, DataFrame
from pyspark.sql.functions import col, sum as _sum, lit, current_timestamp, to_date
from awsglue.context import GlueContext
from awsglue.job import Job
from awsglue.utils import getResolvedOptions
import yaml
```

- Imports PySpark for distributed data processing- Imports AWS Glue libraries for job management and context- Imports YAML for configuration file parsing---

Step 2: SparkContextManager Class

```
class SparkContextManager:
    @staticmethod
    def initialize_spark_contexts():
```

****Purpose:**** Initializes Spark and Glue execution contexts****What it does:****1. Creates SparkContext (entry point for Spark functionality)2. Creates GlueContext (AWS Glue wrapper around SparkContext)3. Gets SparkSession from GlueContext4. Configures Spark for adaptive query execution (performance optimization)5. Gets logger for job monitoring6. Initializes Glue Job object****Returns:**** `(spark, glueContext, job, logger)` tuple---

Step 3: ConfigManager Class

```
class ConfigManager:
    @staticmethod
    def get_job_parameters():
```

****Purpose:**** Loads job configuration from YAML file or Glue job arguments****What it does:****1. Constructs path to `config/glue\_params.yaml` file2. Loads parameters from YAML if file exists3. Attempts to override with Glue job arguments (runtime parameters)4. Handles optional parameters like S3 paths, database names5. Falls back to default job name if no parameters provided****Returns:**** Dictionary of configuration parameters---

Step 4: S3DataReader Class

```
class S3DataReader:
    def __init__(self, spark, logger):
```

****Purpose:**** Reads CSV data from S3 with validation****Key Methods:****##### `\_read\_csv\_internal(path, delimiter, header)` - Internal helper to read CSV files using Spark- Enables schema inference from data##### `\_read\_data\_safe(path, expected\_schema, data\_type, delimiter, header)`****What it does:****1. Reads CSV from S3 path2. Normalizes column names to lowercase3. Checks if DataFrame is empty4. Validates actual columns match expected schema5. Logs record count6. Raises error if schema validation fails****Returns:**** DataFrame or raises exception---

****Step 5: DataQualityTransformer Class****

```
class DataQualityTransformer: def __init__(self, logger):
```

****Purpose:**** Applies data quality rules****Key Methods:****##### ``remove_nulls(df, data_type)``****What it does:****1. Counts initial records2. Removes rows with any NULL values using ``df.na.drop()``3. Removes rows with string literals "Null", "null", "NULL" in any column4. Logs how many records were removed5. Returns cleaned DataFrame##### ``remove_duplicates(df, data_type)``****What it does:****1. Counts initial records2. Removes duplicate rows based on all columns using ``dropDuplicates()``3. Logs how many duplicates were removed4. Returns deduplicated DataFrame---

****Step 6: S3DataWriter Class****

```
class S3DataWriter: def __init__(self, spark, glueContext, logger):
```

****Purpose:**** Writes data to S3 and registers in AWS Glue Data Catalog****Key Methods:****##### ``_write_parquet(df, path)``- Internal helper to write DataFrame as Parquet format to S3- Uses overwrite mode##### ``_write_hudi(df, path, hudi_options)``- Internal helper to write DataFrame using Apache Hudi format- Uses append mode for upserts##### ``write_data_safe(df, path, table_name, database_name, data_type)``****What it does:****1. Writes DataFrame to S3 in Parquet format2. Registers table in Glue Data Catalog3. Makes data queryable via Athena/Glue4. Logs success with record count##### ``write_hudi_scd2(df, path, hudi_config, data_type)``****What it does (SCD Type 2 implementation):****1. Adds SCD Type 2 tracking columns: - ``IsActive``: Boolean flag for current records - ``StartDate``: When record became active - ``EndDate``: When record was superseded (NULL for current) - ``OpTs``: Operation timestamp for conflict resolution2. Configures Hudi options: - Record key (unique identifier) - Precombine field (for deduplication) - Partition field (for data organization) - Operation type (upsert) - Table type (COPY_ON_WRITE for read optimization)3. Writes to S3 using Hudi format4. Enables historical tracking of changes---

****Step 7: OrderSummaryProcessor Class****

```
class OrderSummaryProcessor: def __init__(self, logger):
```

****Purpose:**** Joins customer and order data##### ``create_order_summary(customer_df, order_df)``****What it does:****1. Performs inner join between order and customer DataFrames on ``custid``2. Selects relevant columns from both datasets3. Calculates ``TotalAmount`` = ``priceperunit`` × ``qty``4. Logs record count5. Returns enriched order summary DataFrame---

****Step 8: AggregationProcessor Class****

```
class AggregationProcessor: def __init__(self, logger):
```

****Purpose:**** Generates customer spending analytics##### ``calculate_customer_aggregate_spend(order_summary_df)``****What it does:****1. Groups data by customer attributes and date2. Aggregates total spending per customer per date using ``sum()``3. Creates ``TotalSpend`` column4. Logs record count5. Returns aggregated DataFrame---

****Step 9: Main ETL Execution Function****

```
def main():
```

****Purpose:**** Orchestrates the complete ETL pipeline****Execution Flow:****##### ****9.1 Initialize Components****1. Initialize Spark contexts2. Load configuration parameters3. Initialize Glue job4. Create instances of all processor classes##### ****9.2 Data Ingestion (TR-INGEST-001, TR-INGEST-002)****1. Read customer CSV from S3 with schema validation2. Read order CSV from S3 with schema

```
validation##### **9.3 Data Quality (TR-CLEAN-001, TR-CLEAN-002)**1. Remove NULL values from customer data2. Remove NULL values from order data3. Remove duplicates from customer data4. Remove duplicates from order data##### **9.4 Data Catalog Registration (TR-CATALOG-001, TR-CATALOG-002)**1. Write cleaned customer data to S3 as Parquet2. Register customer table in Glue Data Catalog3. Write cleaned order data to S3 as Parquet4. Register order table in Glue Data Catalog##### **9.5 Order Summary Creation**1. Join customer and order data2. Calculate total amounts##### **9.6 SCD Type 2 Implementation (Optional)**1. If enabled, write order summary using Hudi format2. Enables historical change tracking##### **9.7 Analytics Generation**1. Calculate customer aggregate spending by date2. Write analytics to S3 as Parquet3. Register analytics table in Glue Data Catalog##### **9.8 Job Completion**1. Log success message2. Commit Glue job (marks successful completion)3. Handle errors with logging if job fails---
```

Key Design Patterns

****1. Separation of Concerns****

Each class handles a specific responsibility (reading, writing, transforming, processing)

****2. Error Handling****

Try-except blocks with detailed logging at every step

****3. Configuration Flexibility****

Supports both YAML config files and runtime Glue job arguments

****4. Data Quality Gates****

Validates schemas, removes nulls and duplicates before processing

****5. Catalog Integration****

Automatically registers all outputs in Glue Data Catalog for discoverability

****6. SCD Type 2 Support****

Implements slowly changing dimensions for historical tracking using Apache Hudi---

Data Flow Summary

```
S3 CSV (Customer) ████ █████> Read & Validate ███> Clean (Nulls/Dupes) ███> Write  
Parquet + CatalogS3 CSV (Order) ██████████ █ █ ▼ Join on CustId █ ▼ Order Summary  
(with TotalAmount) █ █ ████████████████████> Write Hudi (SCD Type 2) █ ▼ Aggregate by  
Customer + Date █ ▼ Customer Aggregate Spend Analytics █ ▼ Write Parquet +  
Catalog
```